

Function Tour: Bootstrap and Math Models

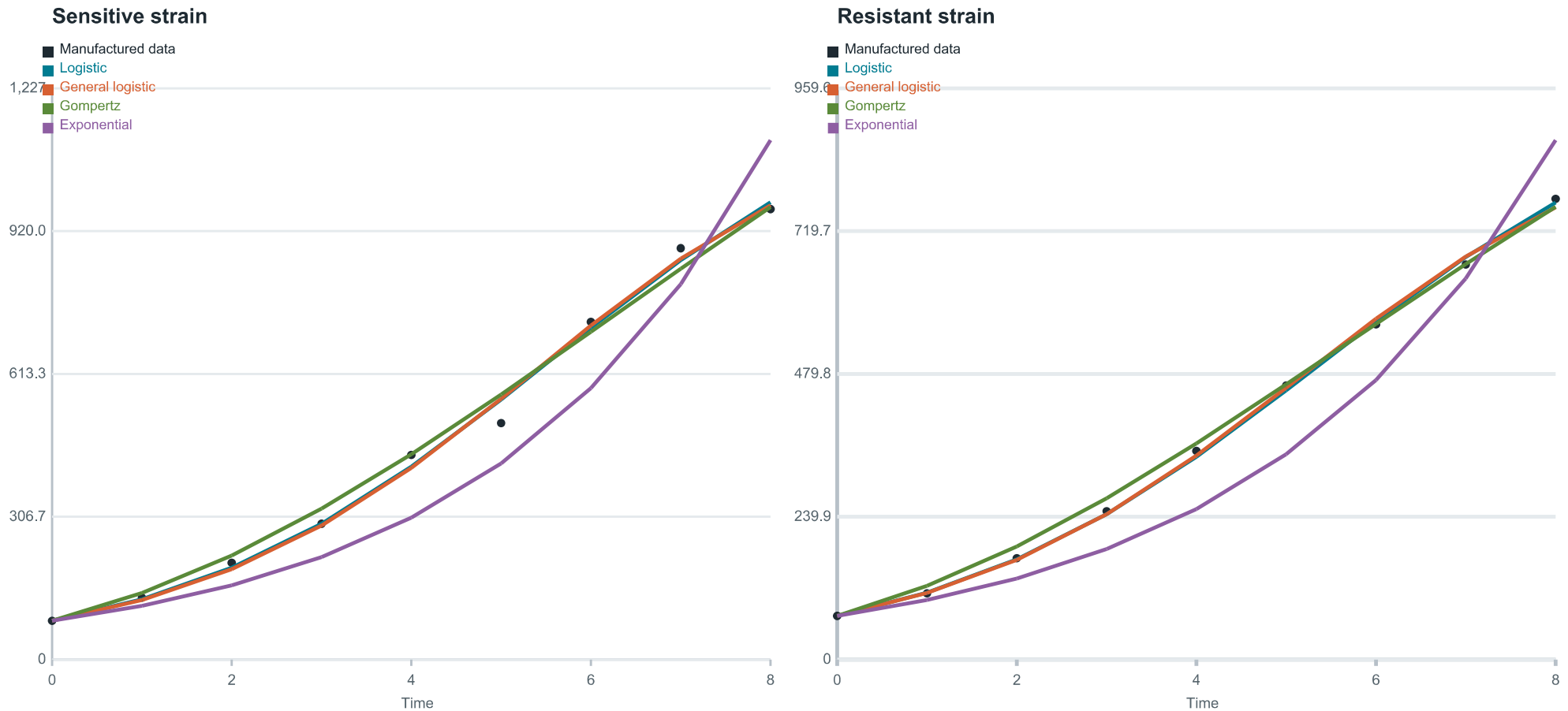
Noisy manufactured data, staged fits, bootstrap uncertainty, model equations, and fitting controls

PDF sequence	What is shown
1. Untreated monoculture	Manufactured logistic data, four candidate fits, BIC, and ideal parameters.
2. Treated monoculture	Prior growth estimates carried forward; logistic plus drug-effect candidates.
3. Untreated coculture	Prior monoculture estimates carried forward; competition-model candidates.
4. Treated coculture	Prior growth, drug, and competition estimates carried forward; combined model candidates.
5. Validation	Sensitivity tests and leave-one-timepoint-out predictions.
6. Bootstrap uncertainty	Residual bootstrap confidence intervals for the untreated sensitive-strain logistic fit.
7. Model reference	Model equations plus fitting controls from basic growth through PK/PD compartments.

Manufactured data and single-condition fits

Manufactured counts for both strains are logistic-growth trajectories with seeded Gaussian measurement noise (2.8% standard deviation). Each strain is fit independently against logistic, Gompertz, exponential, and general logistic growth.

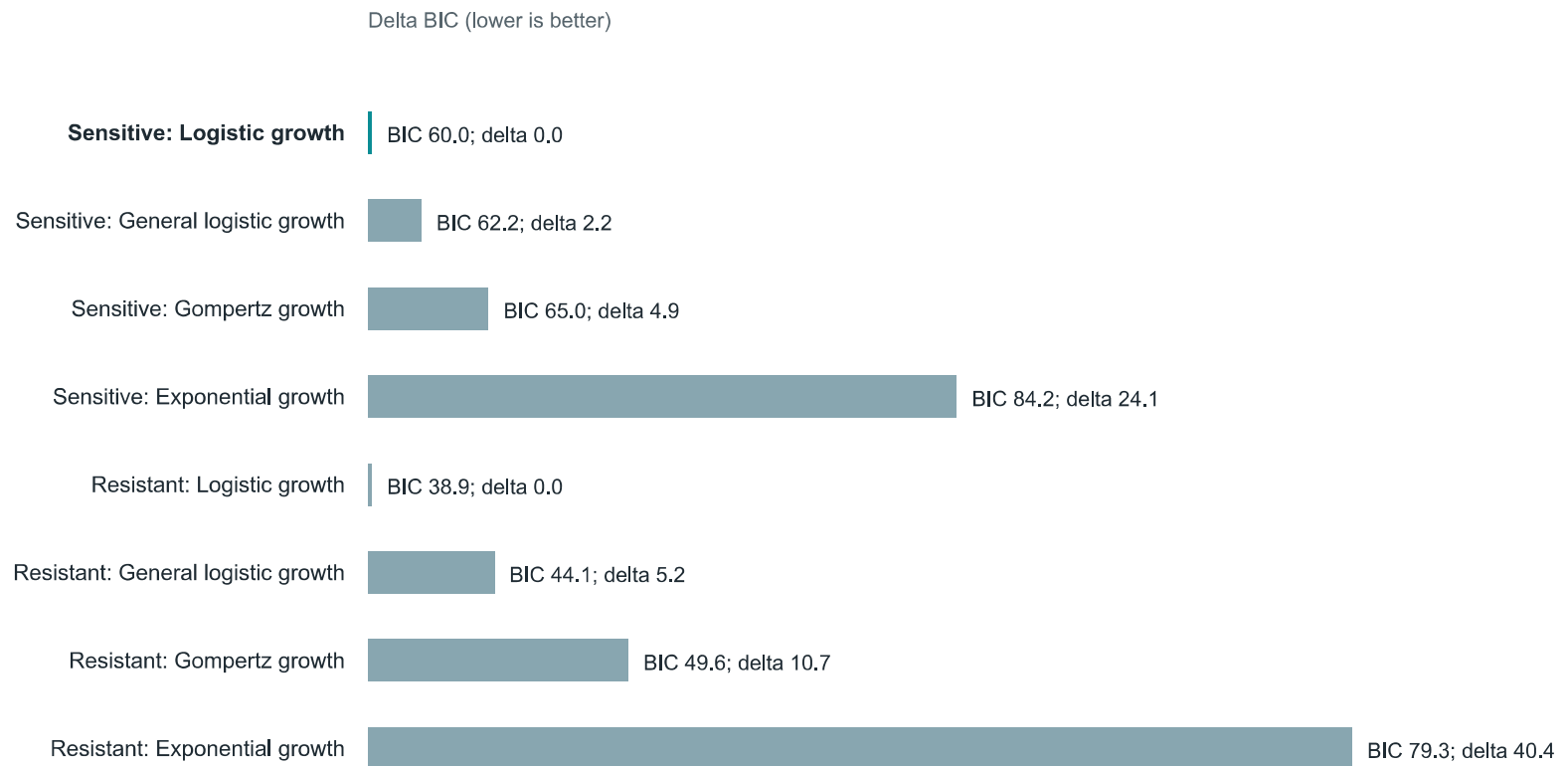
Functions used: `logistic()`, `seeded_random_observation()`, `fit_baseline()`, `optimize_log()`



BIC analysis

The logistic model is the data-generating model in both strains. BIC penalizes the added flexibility of the general logistic model and separates the saturating candidates from exponential growth.

Functions used: `fit_baseline()`, `bic()`



1. UNTREATED MONOCULTURE

Ideal model and parameter handoff

Manufactured truth is shown beside the sequential fit. Inherited values are fixed inputs from the prior stage rather than being re-estimated here.

Functions used: stage_parameter_rows(), PipelineStage parameter handoff.

Role	Model or parameter	Value
Ideal data-generating model	Model	Logistic growth for both strains
BIC-selected model	Model	Logistic growth for sensitive and resistant strains
Manufactured truth	Sensitive r	0.480
Manufactured truth	Sensitive K	1,300
Manufactured truth	Resistant r	0.500
Manufactured truth	Resistant K	900.0
Fitted	Sensitive r	0.482
Fitted	Sensitive K	1,280
Fitted	Resistant r	0.470
Fitted	Resistant K	991.4

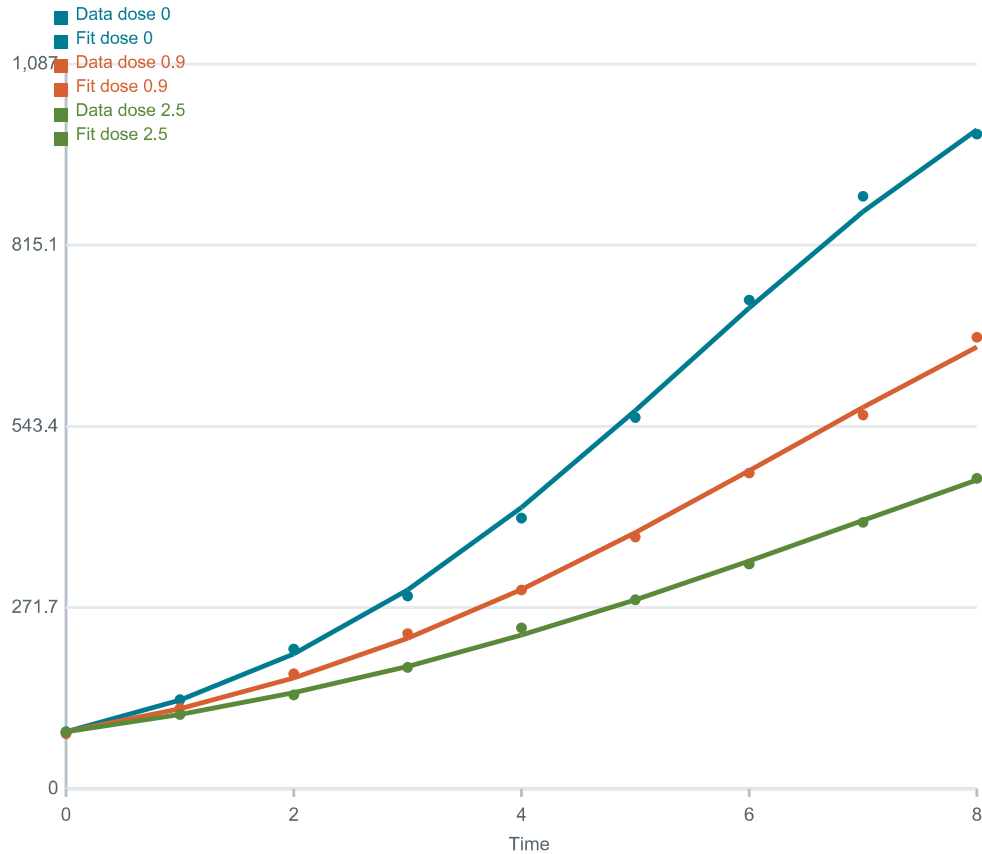
2. TREATED MONOCULTURE

Manufactured drug-modified monoculture fits

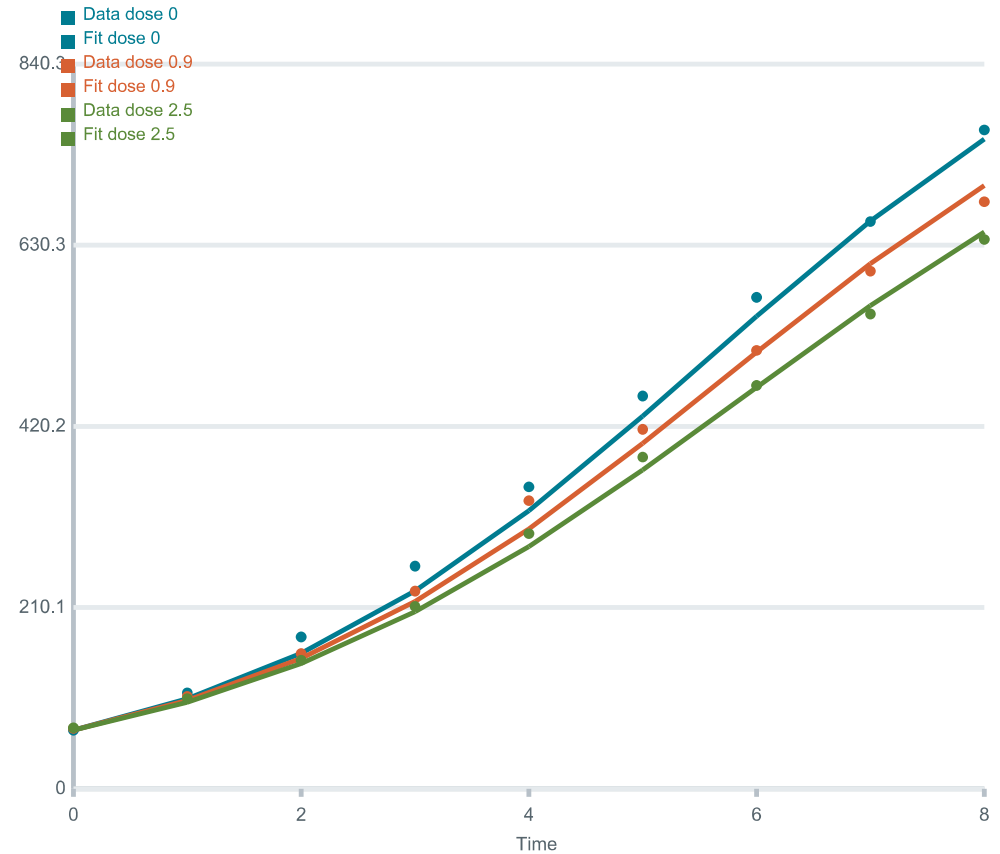
These data begin with the untreated logistic trajectories and add a dose-dependent Hill drug-kill effect. Growth parameters (r and K) are locked to the prior untreated monoculture fit; only drug-effect terms are estimated.

Functions used: `mono_drug_curve()`, `drug_effect()`, `seeded_random_observation()`, `fit_treated_monoculture()`

Sensitive strain



Resistant strain



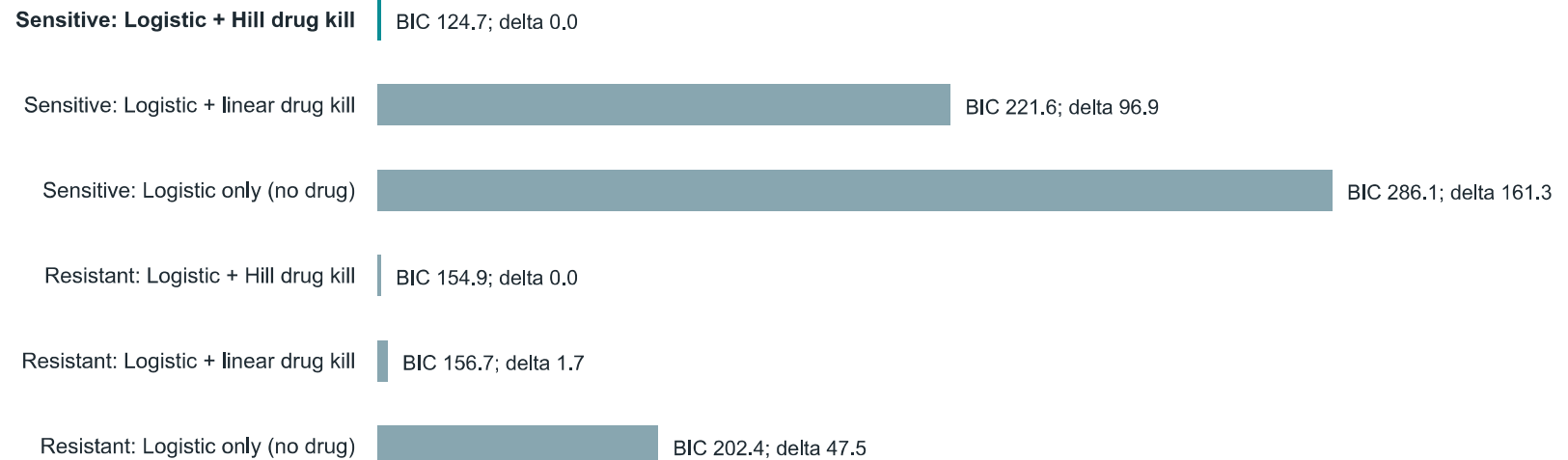
2. TREATED MONOCULTURE

BIC analysis

The Hill drug-kill candidate generated these data and is included in the candidate range. The next page shows the manufactured truth, the sequential fit, and the inherited untreated-growth estimates.

Functions used: `fit_treated_monoculture()`, `bic()`

Delta BIC (lower is better)



2. TREATED MONOCULTURE

Ideal model and parameter handoff

Manufactured truth is shown beside the sequential fit. Inherited values are fixed inputs from the prior stage rather than being re-estimated here.

Functions used: stage_parameter_rows(), PipelineStage parameter handoff.

Role	Model or parameter	Value
Ideal data-generating model	Model	Logistic growth + Hill drug kill
BIC-selected model	Model	Logistic growth + Hill drug kill for both strains
Manufactured truth	Sensitive Emax	0.220
Manufactured truth	Sensitive EC50	0.950
Manufactured truth	Sensitive Hill	1.450
Manufactured truth	Resistant Emax	0.100
Manufactured truth	Resistant EC50	2.300
Manufactured truth	Resistant Hill	1.100
Fitted	Sensitive Emax	0.185
Fitted	Sensitive EC50	0.833
Fitted	Sensitive Hill	2.932
Fitted	Resistant Emax	0.091
Fitted	Resistant EC50	2.952
Fitted	Resistant Hill	1.040
Inherited from prior fit	Sensitive r	0.482
Inherited from prior fit	Sensitive K	1,280
Inherited from prior fit	Resistant r	0.470
Inherited from prior fit	Resistant K	991.4

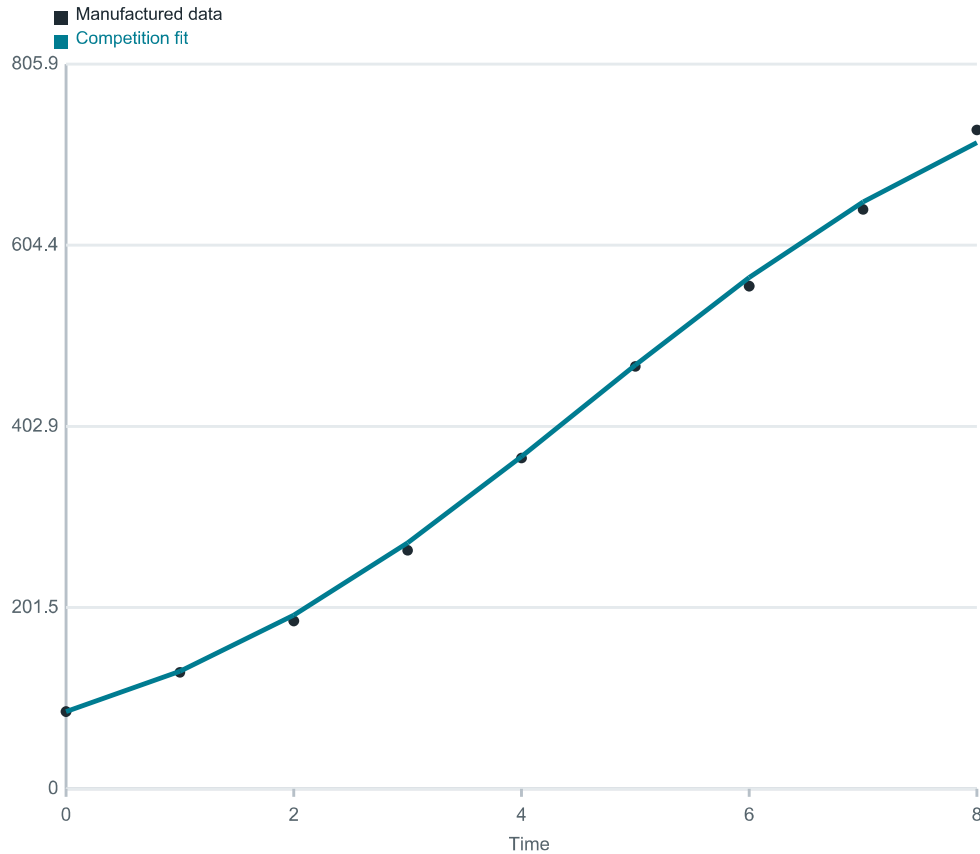
3. UNTREATED COCULTURE

Manufactured competition-modified coculture fit

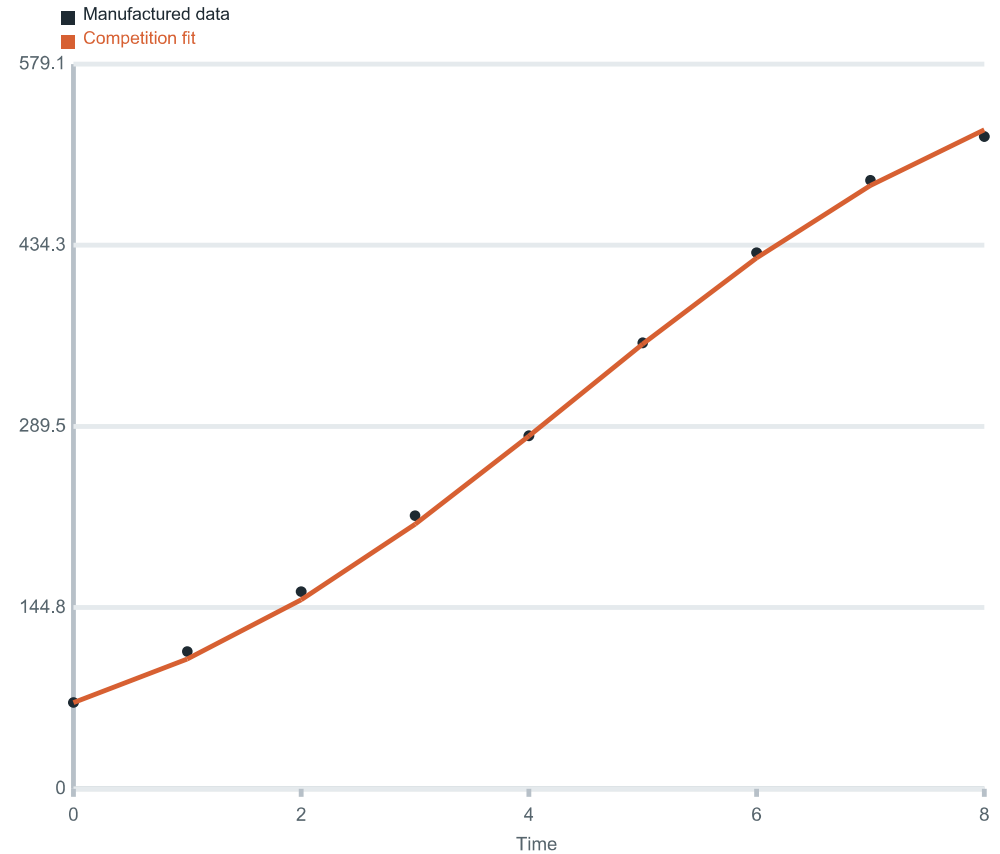
The untreated monoculture data are extended to a two-population logistic system with asymmetric competition. Sensitive and resistant r/K values are inherited from the stage-1 fits, leaving the competition coefficients to be estimated.

Functions used: `coculture_curve()`, `seeded_random_observation()`, `fit_coculture()`

Sensitive population



Resistant population



3. UNTREATED COCULTURE

BIC analysis

The candidate range contains independent growth, symmetric competition, and the asymmetric competition model that generated the data. The selected model exposes both directional competition terms.

Functions used: `fit_coculture()`, `bic()`



3. UNTREATED COCULTURE

Ideal model and parameter handoff

Manufactured truth is shown beside the sequential fit. Inherited values are fixed inputs from the prior stage rather than being re-estimated here.

Functions used: stage_parameter_rows(), PipelineStage parameter handoff.

Role	Model or parameter	Value
Ideal data-generating model	Model	Logistic growth + asymmetric competition
BIC-selected model	Model	Asymmetric competition
Manufactured truth	alpha_SR	0.680
Manufactured truth	alpha_RS	0.420
Fitted	alpha_SR	0.667
Fitted	alpha_RS	0.437
Inherited from prior fit	Sensitive r	0.482
Inherited from prior fit	Sensitive K	1,280
Inherited from prior fit	Resistant r	0.470
Inherited from prior fit	Resistant K	991.4

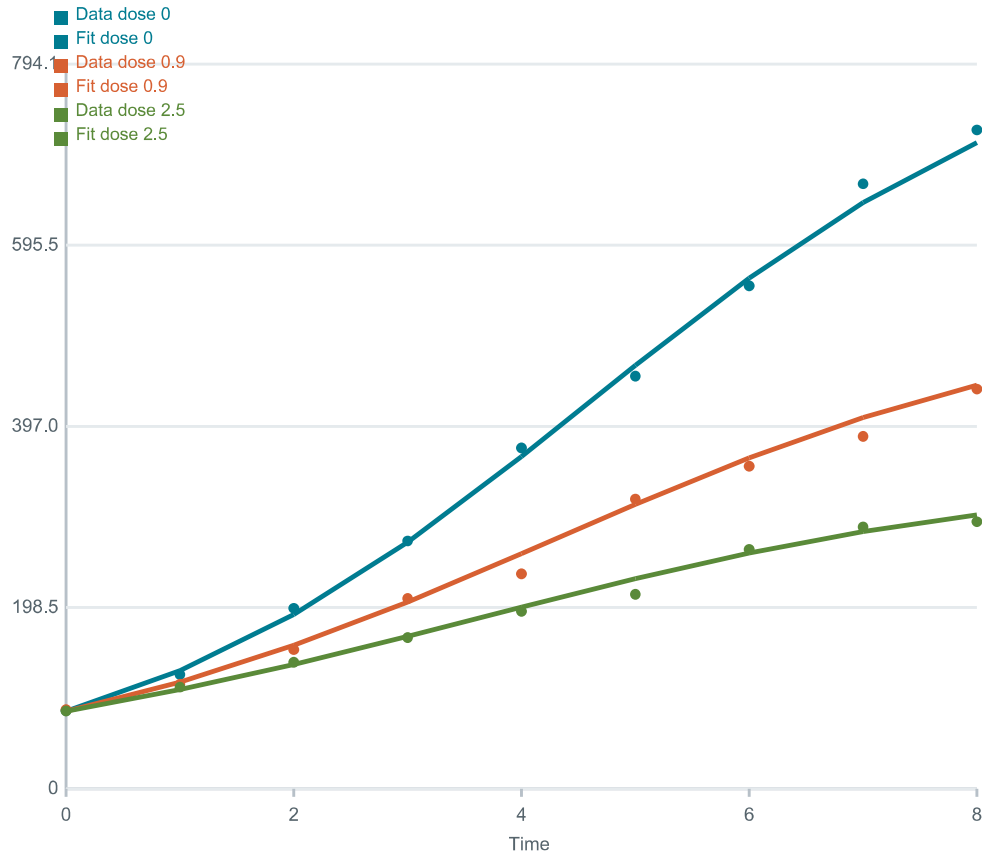
4. TREATED COCULTURE

Manufactured combined drug-and-competition fit

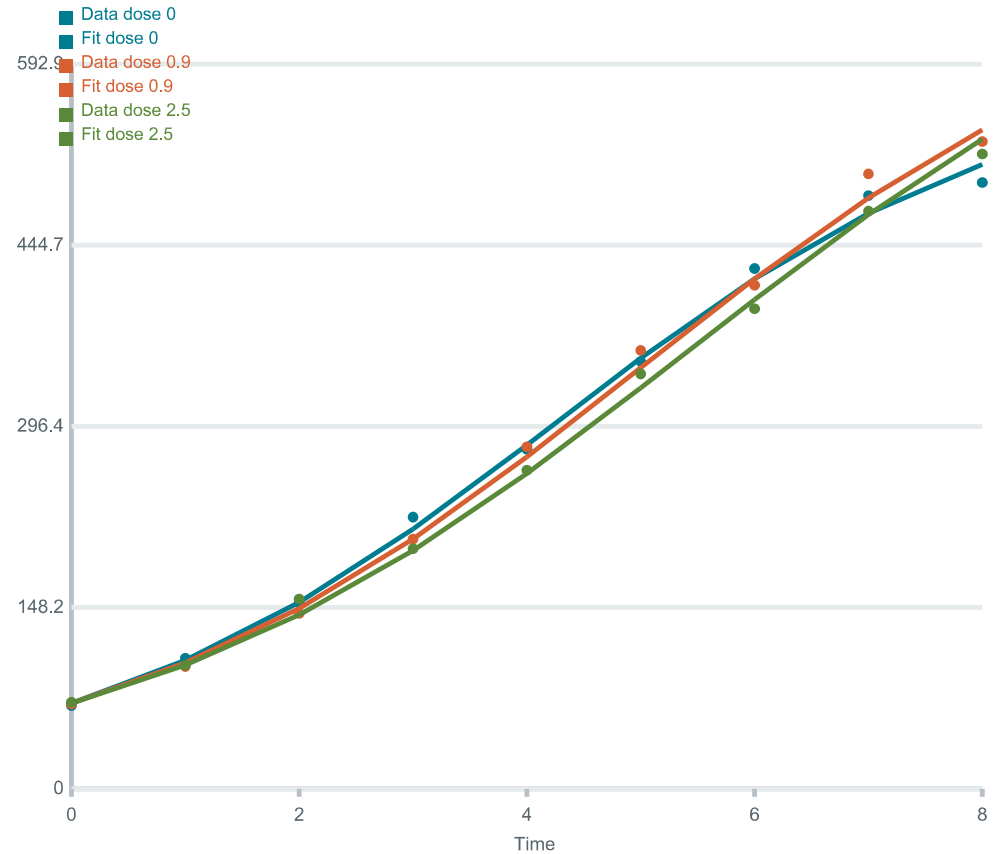
This final data set combines the previous treated monoculture and untreated coculture mechanisms: logistic growth, strain-specific Hill drug response, and asymmetric competition. The fitter inherits r/K , drug parameters, and then estimates the competition terms.

Functions used: `coculture_curve()`, `drug_effect()`, `seeded_random_observation()`, `final_fit()`

Sensitive population



Resistant population

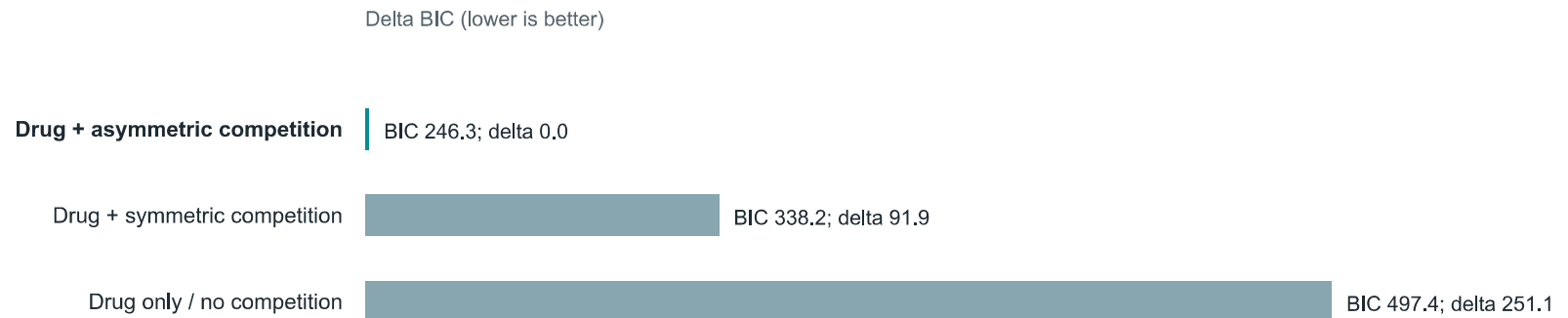


4. TREATED COCULTURE

BIC analysis

The combined asymmetric drug-plus-competition model generated the data and is in the candidate range. Earlier-stage growth and drug estimates are held fixed, so the final stage isolates the interaction terms.

Functions used: final_fit(), bic()



4. TREATED COCULTURE

Ideal model and parameter handoff

Manufactured truth is shown beside the sequential fit. Inherited values are fixed inputs from the prior stage rather than being re-estimated here.

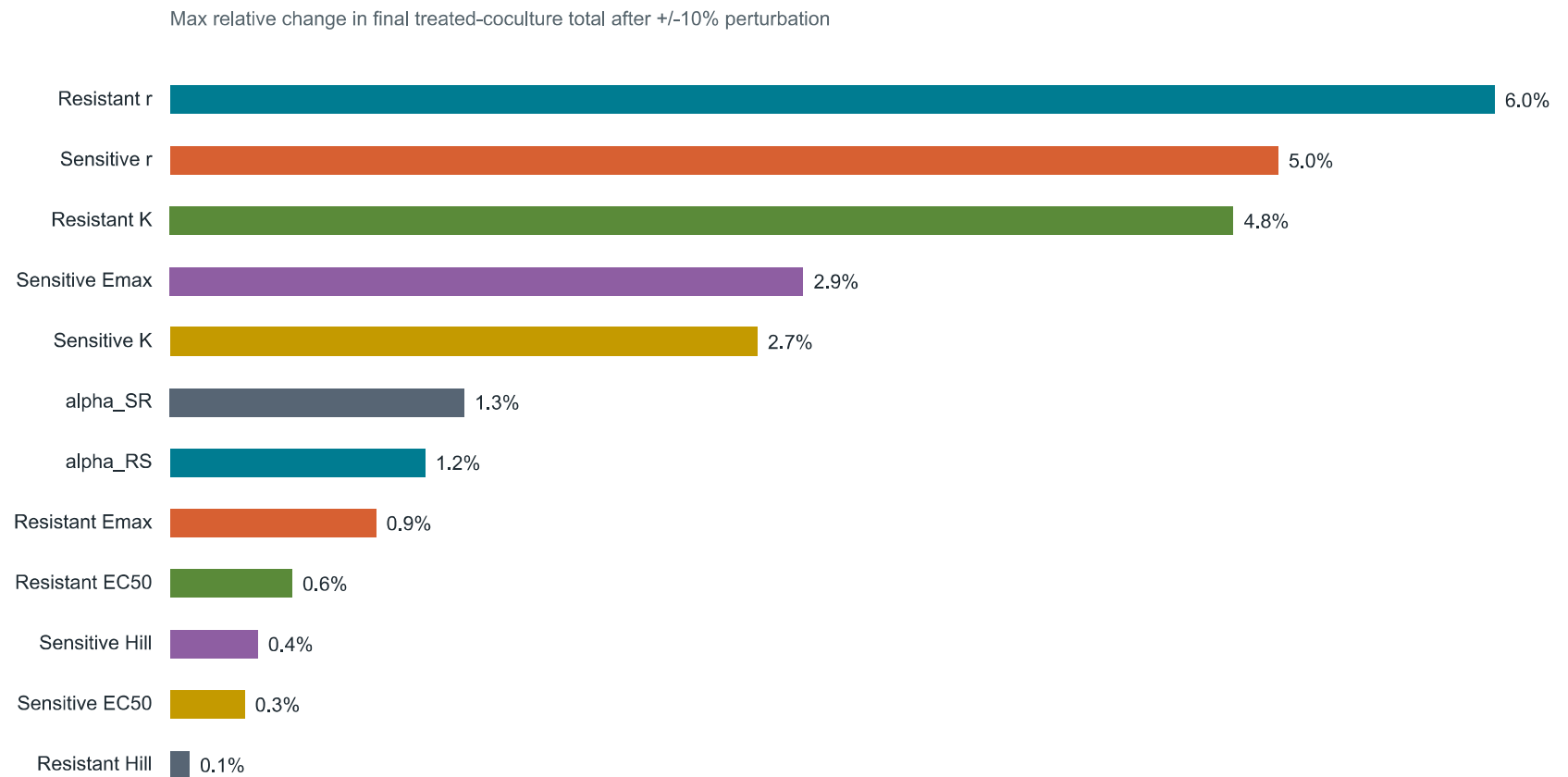
Functions used: stage_parameter_rows(), PipelineStage parameter handoff.

Role	Model or parameter	Value
Ideal data-generating model	Model	Logistic growth + Hill drug kill + asymmetric competition
BIC-selected model	Model	Drug + asymmetric competition
Manufactured truth	alpha_SR	0.680
Manufactured truth	alpha_RS	0.420
Fitted	alpha_SR	0.701
Fitted	alpha_RS	0.484
Inherited from prior fit	Sensitive r	0.482
Inherited from prior fit	Sensitive K	1,280
Inherited from prior fit	Resistant r	0.470
Inherited from prior fit	Resistant K	991.4
Inherited from prior fit	Sensitive Emax	0.185
Inherited from prior fit	Sensitive EC50	0.833
Inherited from prior fit	Resistant Emax	0.091
Inherited from prior fit	Resistant EC50	2.952

Sensitivity tests

One-at-a-time +/-10% perturbations were applied to every inherited and final-stage parameter in the selected treated-coculture model. The bar length is the largest relative change in the final total population at dose 2.5.

Functions used: coculture_curve(), final_total()

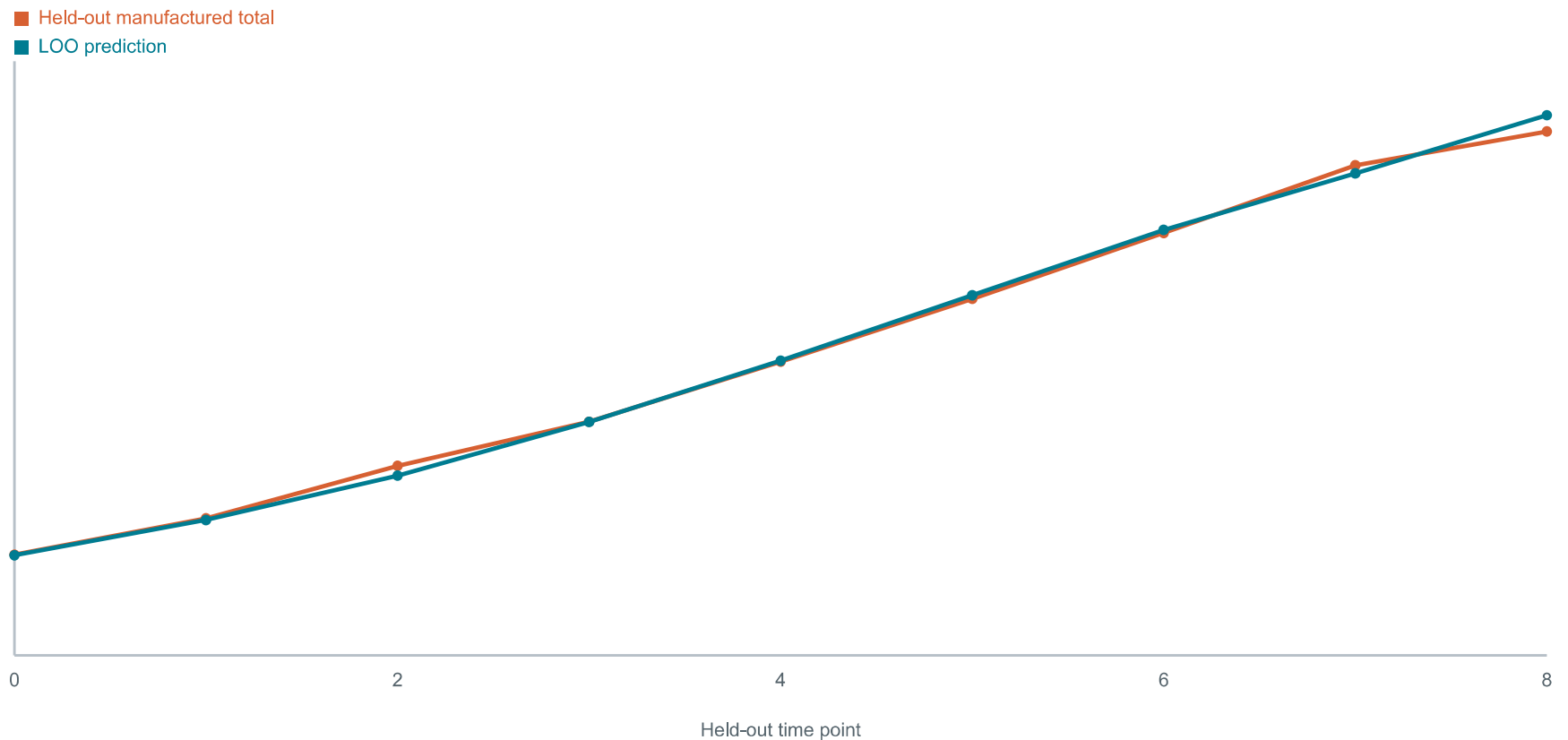


5. VALIDATION

Leave-one-out analysis

For each time point, all observations at that time were withheld across doses and populations. The final treated-coculture competition terms were refit from the remaining observations, then used to predict the omitted high-dose total. Leave-one-timepoint-out RMSE: 10.99.

Functions used: `final_fit()`, `coculture_curve()`

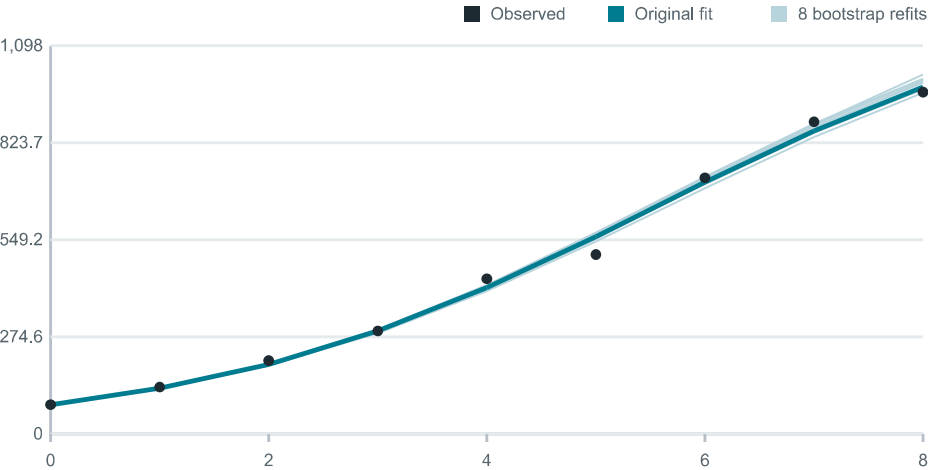


Residual bootstrap example

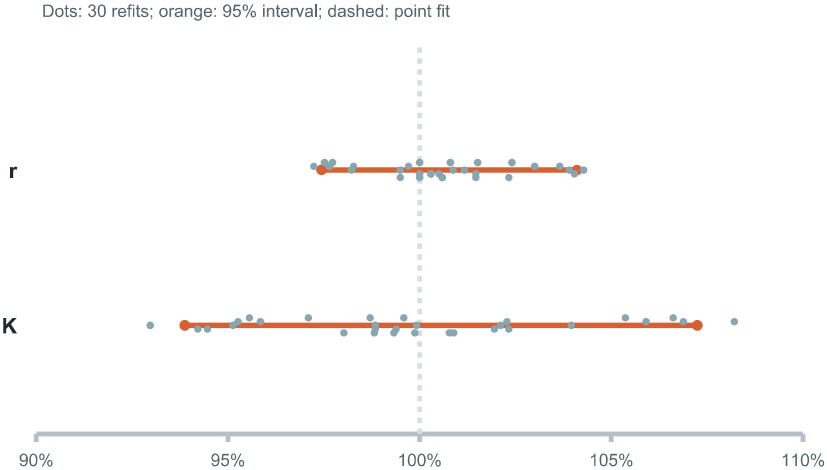
A residual bootstrap is applied to the noisy sensitive-strain untreated logistic fit. The original fit supplies the fitted trajectory and residual pool; 30 reproducible resampled trajectories are refit to form parameter confidence intervals. Package call: `bootstrap_parameter_estimates(times, sensitive_counts, [0.45, 1300.0]; model=build_logistic(), n_bootstrap=30, seed=2026)`.

Functions used: `bootstrap_parameter_estimates()`, `run_single_fit()`

1. Resample residuals and refit



2. Parameter samples form intervals



Parameter	Point fit	Bootstrap mean	Bootstrap SD	95% CI	Successful refits
r	0.482	0.485	0.010	0.470 to 0.502	30
K	1,280	1,280	51.744	1,202 to 1,373	30

7. MODEL REFERENCE

Model examples: basic growth to compartmental drug response

The package can start with a simple logistic curve and scale through coupled populations to mechanistic and compartmental drug-response models. The mathematical forms identify the states and terms being fit.

Functions used: Registry.register_builtin_models!(), Registry.get_model()

Model family	Available function / registry model	Fitted mathematical form	What it captures
Basic growth	build_logistic(), build_gompertz(), build_exponential()	$dN/dt = r N (1 - N/K)$; $dN/dt = r N \log(K/N)$; $dN/dt = r N$	Single-population growth from exponential to carrying-capacity-limited curves.
Generalized growth	theta_logistic_hill_inhibition, theta_logistic_hill_kill	$dN/dt = g(N; r, K, \theta) - E(d) N$	Theta-generalized logistic growth with dose-dependent Hill inhibition or kill.
Death and kill	logistic_linear_kill, logistic_growth_with_death, gompertz_growth_with_death	$dN/dt = g(N) - k N$	A baseline growth model plus a linear death or treatment-kill term.
Coculture	lotka_volterra_competition, lotka_volterra_cooperation	$dS/dt = rS S[1 - (S + aSR R)/KS]$; $dR/dt = rR R[1 - (R + aRS S)/KR]$	Coupled sensitive/resistant populations with competitive or cooperative interactions.
Drug + coculture	lotka_volterra_hill_competition, sensitive_resistant	$dS/dt = gS(S, R) - ES(d) S$; $dR/dt = gR(S, R) - ER(d) R$	Population interactions combined with strain-specific drug response and resistance dynamics.
PK/PD and compartments	pkpd_inhibition, transit_chain_erlang	$dC/dt = k_{in} d - k_{el} C$; $dN/dt = g(N) - E(C) N$; $N \rightarrow X1 \rightarrow X2 \rightarrow X3$	Drug concentration compartments or a transit chain for delayed drug effects.
Mechanistic extensions	damage_repair_arrest, adaptive_ic50	$N \rightarrow D \rightarrow A$; $IC50(t) = IC50_0 \exp(k_{adapt} t)$	Damage/repair/arrest states or adaptive drug sensitivity over time.

8. FITTING CONTROLS

How fitting can be adjusted

This reference summarizes the principal package controls for moving from a simple fit to a constrained, weighted, multi-condition staged analysis.

Functions used: `Workflow.fit()`, `rank_models()`, `run_staged_pipeline()`, `run_single_fit()`

Adjustment	Function or keyword	What it changes
Choose and compare models	<code>Registry.get_model()</code> , <code>rank_models()</code>	Select candidate models and rank them by BIC.
Define experimental groups	<code>build_conditions()</code>	Build fit conditions from culture, strain, dose, replicate, or other metadata.
Share parameters	<code>Workflow.fit(...; shared_params=...)</code>	Require selected parameters such as r or K to be common across conditions.
Fix a prior estimate	<code>Workflow.fit(...; fixed_params=...)</code>	Hold earlier-stage values fixed while estimating the new mechanism.
Tie parameters	<code>Workflow.fit(...; tie_constraints=...)</code>	Constrain one parameter to match another within a joint fit.
Use staged handoff	<code>PipelineStage(... parameter_sources=...)</code> , <code>run_staged_pipeline()</code>	Pass untreated growth estimates to drug fits, then pass growth/drug estimates to coculture fits.
Control optimization	<code>n_starts</code> , <code>maxiters</code> , <code>seed</code> , <code>top_k</code>	Adjust multi-start search effort, reproducibility, and retained candidate fits.
Weight and solve	<code>weighted</code> , <code>FitCondition.error</code> , <code>reltol</code> , <code>abstol</code>	Weight residuals by measurement error and tune the ODE solver precision.
Direct one-curve fit	<code>run_single_fit(...; p0, bounds, fixed_params, solver)</code>	Set initial guesses, bounds, fixed parameters, and solver for a focused fit.

Preserve the bootstrap interval; add identifiability checks before interpretation

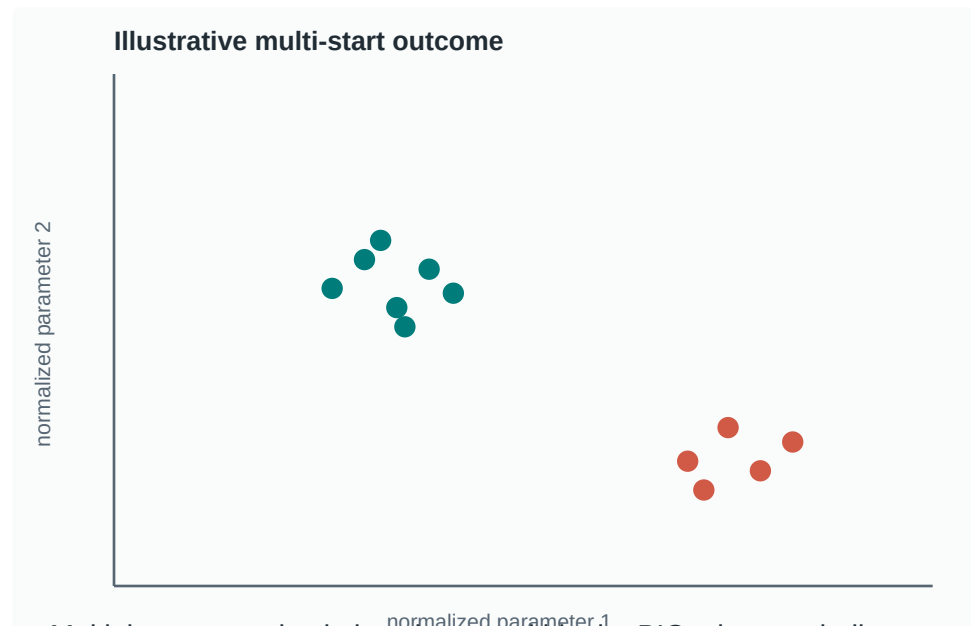
This appendix follows the original 18 pages unchanged. Section 6 still provides residual-bootstrap 95% confidence intervals; the v0.5.0 tools ask whether the model and measurement design can support a unique biological interpretation.

Existing evidence	New check	Reason
Bootstrap 95% CI	bootstrap_joint_fit	refit full multi-series models under residual or parametric noise
Sensitivity analysis	fisher_information	test local parameter-direction information and conditioning
Best fitted solution	generate_multistarts	detect distinct parameter clusters with comparable fit quality
Mechanistic model	structural_identifiability	separate symbolic global/local proof from numerical stability

- The bootstrap interval remains necessary: it describes finite-sample uncertainty around the chosen model.
- Practical and structural identifiability answer different questions and should be reported alongside the bootstrap result, not instead of it.

Use broad starts, profile likelihoods, and bootstrap success to gate complex fits

The numerical report is deliberately conservative: a low SSR, a narrow bootstrap interval, or a full-rank local matrix alone is not enough to establish a reliable mechanistic parameter estimate.



```
config = IdentifiabilityConfig(
    [:growth, :capacity, :drug_damage], bounds;
    n_starts=40, start_scale=:log,
    profile_points_per_side=12, bootstrap_replicates=200,
    bootstrap_method=:parametric)
report = practical_identifiability_report(
    model, datasets, u0, p0; config=config)
```

- Multiple separated solution clusters within the BIC tolerance indicate parameter confounding.
- Fisher rank and condition number identify local weak directions; profiles test whether the data bracket a parameter.
- The report includes bootstrap success rate and retains failed refits so uncertainty is not hidden.

For a global or local claim, define exactly what was measured

Structural analysis uses `StructuralIdentifiability.jl` with the same state-to-assay mapping and known inputs as the numerical fit. It never infers a proof from an optimizer result.

```
map = ObservationMap(
    "drug_macrophage_four_state",
    [:S, :D1, :D2, :M],
    [:viable_cells, :recoverable_damage, :terminal_damage, :macrophages])
validate_observation_map(map, dataset_specs)
result = structural_identifiability(symbolic_ode; mode=:global)
```

- In a four-population model, directly measuring viable, damaged, terminal-damage, and macrophage states is much more informative than measuring total viability alone.
- Drug concentration is a known input in the symbolic ODE; it is not an unobserved estimated state.
- Use `synthetic_recovery_benchmark` at the actual sampling times before drawing conclusions from sparse real data.

Screen the full biological parameter range, not only the best-fit neighborhood

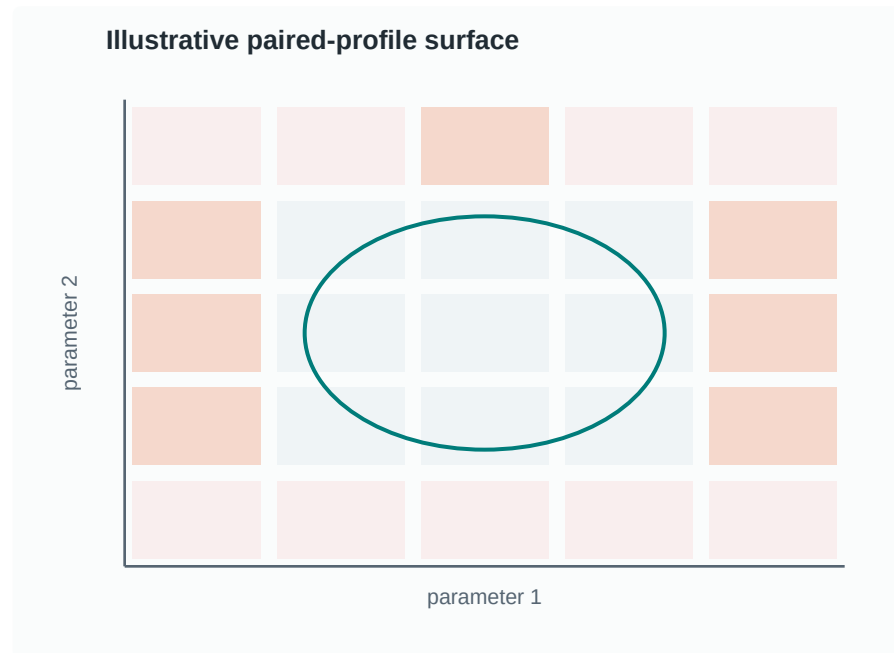
`global_sensitivity_analysis` uses Latin-hypercube samples and partial rank correlations (PRCCs) across every measured series and time point.

```
global = global_sensitivity_analysis(  
    model, dataset_specs, u0;  
    bounds=bounds, parameter_names=parameter_names,  
    n_samples=400, scale=:log)  
global.summary # mean and maximum absolute PRCC  
global.pointwise # PRCC by series and time
```

- A large absolute PRCC means a parameter strongly changes an observed output across its allowed range after accounting for other sampled parameters.
- A low PRCC does not prove irrelevance: non-monotone effects and parameter interactions can still matter.
- Use this analysis to decide which parameters, time points, doses, or direct measurements are most worth improving.

Expose tradeoffs that one-parameter confidence intervals can hide

`paired_profile_likelihood` fixes two parameters together on a grid and re-fits every other parameter at every grid point.



```
paired = paired_profile_likelihood(  
    model, dataset_specs, u0, p0;  
    bounds=bounds, parameter_names=parameter_names,  
    pair=( :drug_damage, :macrophage_killing),  
    points_per_side=10)  
paired.accepted_region
```

- The default threshold is 5.991, the two-parameter 95% chi-square cutoff.
- A diagonal accepted region means the two biological effects can trade off while preserving the fit.
- A region touching a bound means the experiment has not contained the combined uncertainty.

Analyze sensitive, resistant, and treatment groups in one pooled objective

`hierarchical_joint_fit` estimates central shared parameters and partially pooled group-specific effects. It is not an average of separately fitted parameter vectors.

Component	Meaning	Why it helps
central parameters	one shared biological baseline	all groups inform shared growth or capacity
varying parameters	group deviations on log scale	allows resistant or treated groups to differ
random_effect_sd	fixed pooling strength	prevents small groups from producing extreme unsupported estimates

```
hierarchical = hierarchical_joint_fit(
    model, groups, p0; bounds=bounds,
    parameter_names=parameter_names,
    varying_parameters=[:growth, :drug_damage],
    random_effect_sd=[0.35, 0.50])
```

- Each group supplies name, `dataset_specs`, and `u0`. All observations contribute to the same data SSE and pooled BIC.
- Use `group_parameters` to report central estimates, each group estimate, and its log-scale deviation.